

Mid Term Examination Winter 2020

Branch: 5th Sem, B.Tech. CSE

Course: Operating Systems (CSL309)

Duration: 60 minutes

Max. Marks: 25

Note: Answer the questions as per the answer script template and scan as per the instructions already provided.
State clearly any assumptions and reasoning for the answers, wherever necessary.

Q1. OS requires certain support from the H/W in order to effectively implement “Limited Direct Execution” mechanism for CPU and memory virtualization. Briefly state any two distinct support/aides required from the H/W and its purpose/reason for each of the following cases: [3 x 2 = 6 marks]

- (a) Specific to CPU virtualization mechanism
- (b) Specific to Memory virtualization mechanism
- (c) Common to both CPU and Memory virtualization mechanism

Ans: a) CPU virtualization requires HW support as -

- 1) Timer interrupt for switching CPU between processes
- 2) ability to save/restore the user/process context to and from kernel stack during context switch or system call trap

b) Memory virtualization requires – (any two)

- 1) registers like Base/Bound or Page table address – for storing the addresses of segments, page table etc. so as to allow dynamic relocation of address space
- 2) Circuitry in MMU for address translation and validity check
- 3) In case of paging, TLB is needed to for caching address translations to improve access time

c) Common to both (any two)

- 1) Mode bit/register for enabling dual mode operation/security
- 2) raising Interrupts/Traps for handling various events/situations like system call
- 3) Set of privileged instructions for OS to do sensitive stuff etc.

What is not accepted as correct answer?

- I. Writing the answers in the wrong category like MBR/dual mode operation or raising trap/interrupts as answer to part a) i.e specific to CPU virtualization is not considered.
- II. Not distinct answers like base and bound registers is one category and not considered as two different H/W support. Similarly protection bits etc are already part of TLB or page table and are not considered as distinct answer
- III. The question asked for specific/additional H/W support for CPU and memory virtualization. So any H/W part that is already a regular part of a normal system like general registers, memory etc. are not considered here. So answers like page table or interrupt descriptor table are not considered because they are stored in regular memory which is not an additional support from H/W for the purpose of LDE.

Q2. Briefly state four distinct reasons (or advantages) why modern OS support/implement memory virtualization. [2 marks]

Commented [PK1]: Marking scheme -
Total 6 points each 1 marks. 0.5 marks for mentioning the type/name of the support and 0.5 for the purpose/reason.
0 marks for cases involving copying from others

Ans: 1) Efficient and easy abstraction to user as it free's the application programs from having to manage shared memory space and leave to the OS the burden of managing it.
 2) Better Protection due to memory isolation
 3) Despite of protecting each process address space, once can share certain portions like code etc.
 4) Being able to run processes even if they require more memory than might be physically available.

Commented [PK2]: Marking scheme – 0.5 marks for each reason

What is not accepted as correct answer?

1. Helps for Speed gain
2. Simply writing Multiprogramming implementation without any details like easy abstraction/transparency or being able to run many with less RAM etc.

Q3. Consider a multilevel feedback queue scheduler with three queues, numbered Q0 to Q2 (Q0 being the highest priority queue) and time quantum of 4 ms, 8 ms and 16ms respectively. If a process arrives in a higher priority queue, it will preempt any process in lower queue. Suppose that the processes arrive for executing as per following workload table (numbers are in milliseconds). Assume no I/O and no priority boosting. Let N (in the table) be the second last digit of your enrollment no % 5. For ex. If enrollment no is BT18CSE084 then N=3(8%5). [2+1+1+1=5 marks]

Commented [PK3]: The rubrics in the Gradescope will tell the details of marking.

Process ID	Arrival Time	CPU Time
P0	0	10 + N
P1	8 + N	05 + N
P2	12 + N	10 – N
P3	15 + N	08 – N

(a) Draw the scheduling of processes with time on X axis and different Queue's on Y axis (as discussed in the class & book for MLFQ scheduling). A sample representation is shown below:

Q0									
Q1									
Q2									
Time ->	0 – T1	T1 - T2	...	...					

Also compute the following metrics: (b) The average turnaround time (c) The average response time and (d) The average waiting time

Solution: Run this script (<http://pages.cs.wisc.edu/~remzi/OSTEP/Homework/HW-MLFQ.tgz>) and verify your answer. See the readme for the info of the options and output by the script. To get answer for our problem, you can run command like this (ofcourse with the job list as per your roll no):

```
$>./mlfq.py -l 0,10,0:8,5,0:12,10,0:15,8,0 -Q 4,8,16 -c
```

The given script does not calculate average waiting time. You can modify the script to calculate waiting time of each process using below formula and then calculate its average.

Waiting time = Turnaround time / CPU time.

Q4. One day Govind had an inspiration. He observes that most programs have majority of their code/data at the beginning of the address space. For his homegrown custom OS, Govind decides that he is going to implement paging mechanism using a two-level page table structure with the following twist: The first half of the page table entries in the first level page table (Page Directory) directly maps physical pages, and the second half of the entries map to the second level page tables as normal. Call the first half entries as fast, and the second half as normal. For the following questions, assume that the page size is $2 \times (1+N)$ KB, size of PTE is 4 bytes and that all the page tables fit into a single page. Where N is same as described in Q2 above (i.e the second last digit of your enrollment no. % 5). [1 x 5=5 marks]

- (a) How many virtual pages are fast pages?
 - (b) How many virtual pages are normal pages?
 - (c) What is the maximum size of an address space in bytes (use exponential notation for convenience, e.g., 2^3)?
- If TLB fetch time is T (with hit ratio as H) and Memory access time is M , what is the time taken for a memory reference (assume all referenced pages in memory) whose entry is in:
- (d) fast page (e) normal page?

Answer: if $N=1$, page size = 4KB and the solution will be as below (accordingly change it for your case)

a) $4KB/4 = 1024$ PTEs

$1024/2 = 512$ PTEs $\Rightarrow 512$ or (2^9) pages are fast

b) The remaining 512 PTEs refer to second-level page tables. Each second-level page table has 1024 PTEs, so:

$2^9 \times 2^{10} = 2^{19}$ pages are normal

c) The size of the address space is the total number of pages times the size of each page: $(2^9 + 2^{19}) \times 2^{12} = 2^{21} + 2^{31}$ bytes

d) EAT for fast pages = $T \cdot H + (1-H) \cdot (T+M) + M = T + (2-H) \cdot M$

e) EAT for normal pages = $T \cdot H + (1-H) \cdot (T+2M) + M = T + (3-2H) \cdot M$

Commented [PK4]: 1 mark for each correct answer and 0.5 mark deduction per mistake. The rubrics in the Gradescope will tell the details of marking.

Q5. A system has given specifications: its virtual address is 32 bits long, its page size is 4KB, its physical memory has 1024 page frames and its TLB can store 32 page entries. In this problem, we will try to understand what happens while running some test code. Before running the test code, though, we first run the following *initialization code*.

```
int k = 0;
void *store = malloc(NUM_PAGES * PAGE_SIZE);
void *ptr = store;
while(k++ < NUM_PAGES) {
    (int *) *ptr = 0; // initialize first value on each page
    ptr += PAGE_SIZE;
}
```

The initialization code allocates some amount of memory and sets the first integer of each page to 0. Assuming malloc() returns page-aligned memory in this example. After running the initialization code, we run the following *test code*:

```
ptr = store;
k = 0;
while(k++ < NUM_PAGES) {
    int x = (int *) *ptr; // load value pointed to by ptr
    ptr += PAGE_SIZE;
}
```

We are only interested in memory accesses to the malloc'd region through ptr and ignore all other memory accesses (i.e., ignore instruction fetches and stores to x). We further assume that memory and TLB start from a clean state before we run the initialization code. Given that LRU replacement policy is used whenever such a policy might be needed by either hardware or software:

Fill the table with no. of TLB hits, misses and page faults that occur during the test code when: [3 marks]

	TLB hits	TLB misses	Page Faults
(a) NUM PAGES is 32?	32	0	0
(b) NUM PAGES is 1024?	0	1024	0
(c) NUM PAGES is 2048?	0	2048	2048

Commented [PK5]: Total 1.5 marks (0.5 for each part if all 3 values correct)

Reason: The workload just loops over some pages accessing each one once. Thus, the second time through the loop with a small number of pages (less than or equal to the TLB size, for example) just yields TLB hits. When it exceeds that because of sequential access LRU will result in all TLB misses but till no. of pages are less than memory frames, there wont be any page faults. With a large number of pages, and LRU replacement, nothing we want is ever in the cache or TLB, hence all misses/page faults.

Commented [PK6]: 1.5 for reasoning – 0.5 for reason for each part

If TLB lookup time is T, memory access time is M and a disk access time is D, fill in the blanks with the runtime of the test code when: [3 marks]

(d) NUM PAGES is 32?	$32 * (T + M)$
(e) NUM PAGES is 1024?	$1024 * (T + 2M)$
(f) NUM PAGES is 2048?	$2048 * (T + 2M + D)$

Commented [PK7]: 0.5 marks for each correct answer – total 1.5

Reason: A TLB hit means a TLB access and load to memory which just costs (T+M). On TLB misses we need to consult memory twice, once for the page table (minimally) and once for the actual memory load, hence (T+2M) for all TLB miss is pages are in memory. Finally, each page fault costs us an additional disk access

Commented [PK8]: 1.5 for reasoning – 0.5 for reason for each part

(g) How could you use the test code above to learn about the H/W (size of TLB & Memory) of a new machine of which you know very little about? [1 marks]

Just looking for something simple here. You could basically time how long an iteration of the loop lasts. If it lasts something like NUM PAGES times memory access time, those are TLB hits; if it lasts twice that, TLB misses, and thus you can know how big the TLB is. One more big jump occurs when NUM PAGES is finally bigger than memory and causes lots of (very slow) disk accesses

Commented [PK9]: 1 marks. Marks may be deducted if explanation not given/clear

----- XXX -----